Check for updates

Computer Applications in Engineering Education

WILEY

RESEARCH ARTICLE OPEN ACCESS

How to Visualize Computational Mechanics: Animating Finite Elements, Continuum Mechanics, and Tensor Calculus

Moritz Flaschel

Institute of Applied Mechanics, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany

Correspondence: Moritz Flaschel (moritz.flaschel@fau.de)

Received: 24 January 2026 | **Revised:** 26 March 2026 | **Accepted:** 5 April 2026

Keywords: 3D animations | computational mechanics | continuum mechanics | didactic methods | finite element method

ABSTRACT

Computational mechanics is inherently complex, as the underlying physical processes evolve in three spatial dimensions and time. Thus, intuitive visualizations in traditional lecture notes or on blackboards are challenging. This motivates the use of animation software to illustrate time-dependent three-dimensional phenomena, which complement conventional lectures and exercises and help to clarify otherwise abstract theoretical concepts. The mathematical foundations of computational mechanics—such as weak formulations in finite element analysis, the interpretation of the deformation gradient as the Jacobian of the motion mapping, or the tensorial nature of stress and strain—are often difficult for students to grasp intuitively. In response to these challenges, we discuss didactic approaches for conveying key concepts in computational mechanics, with a focus on continuum mechanics, tensor calculus, and the finite element method. We present concrete examples demonstrating how these concepts can be visualized using open-source software such as Matplotlib, Manim, and Blender. We summarize insights gained from the development of a series of educational videos on computational mechanics and discuss the didactic impact of visualization on student comprehension.

1 | Introduction

Computational mechanics encompasses numerous abstract concepts that often resist intuitive explanation. As a result, students commonly encounter conceptual hurdles, asking questions such as: *How can the weak formulation, which is central to finite element analysis, be understood in a visual and intuitive manner?* or *Why must stresses and strains be described by tensors rather than mere matrices?* Animated visualizations, when combined with carefully designed didactic concepts, offer a promising strategy for addressing these challenges.

It is well established that visualization plays a key role in enhancing our understanding of abstract concepts in mathematics and mechanics [1–8]. Since antiquity, mathematical and mechanical concepts have been visualized using diagrams and geometric constructions. While ancient Greek mathematicians are frequently portrayed as illustrating geometric concepts through drawings in sand [9], modern education relies on visualizations on blackboards and in lecture notes. With recent advances in computational tools and visualization technologies, however, these classical representations are increasingly being complemented by more advanced visualization techniques [10]. For instance, transforming a sequence of static visualizations into an animation enables an intuitive illustration of concepts such as motion and deformation. Moreover, such animations can be programmed to depend on adjustable parameters, allowing for interactive exploration by users, as implemented in software environments, such as Jupyter Notebooks [11], Mathematica [12], or Desmos [13]. In recent years, interactive mathematical visualization has progressed toward the integration of virtual and augmented reality, facilitating immersive representations of mathematical structures and phenomena [14]. Finally, there have been efforts to facilitate teaching through the use of video games [15]. In the context of computational mechanics, for example, the application *Truss me!* [16] has been developed to support

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). *Computer Applications in Engineering Education* published by Wiley Periodicals LLC.

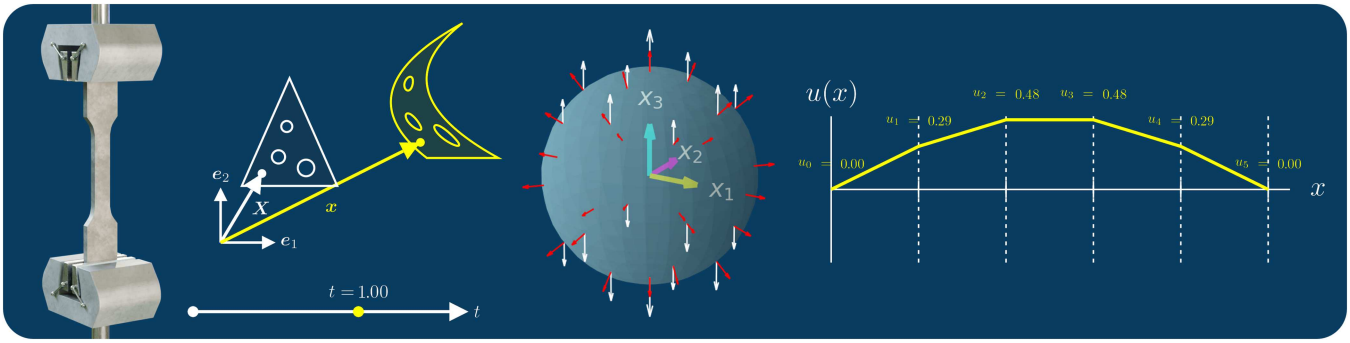


FIGURE 1 | Graphical abstract. Animation tools such as Matplotlib, Manim, and Blender are used to visualize concepts from continuum mechanics, tensor calculus, and the finite element method.

the teaching of structural mechanics concepts, and more recently, Beck et al. [17] presented an elasticity solver implemented within the video game *Minecraft*.

In this contribution, we focus on animated visualizations in the field of computational mechanics and discuss how they can be effectively combined with carefully designed didactic concepts (Figure 1). While animated visualizations are less interactive than virtual and augmented reality environments or video games, they offer the important advantage of broad accessibility. In particular, they require no specialized hardware, software installation, or user setup, and can be readily accessed by a wide audience once made publicly available online. Such videos may be used prior to formal instruction to sensitize students to upcoming topics, or after lectures to reinforce and deepen visual understanding of previously introduced theoretical concepts.

In this regard, the aims of this contribution are twofold. First, we provide practical guidelines for creating graphical animations suitable for applications in computational mechanics. We discuss the three open-source animation software tools Matplotlib [18], Manim [19, 20], and Blender [21] in Section 2 and demonstrate, through concrete examples, how they can be effectively leveraged in computational mechanics. Second, in Section 3, we outline didactic strategies in computational mechanics to effectively integrate and utilize the mathematical animations described above.

We note that the figures in this paper do not convey motion as effectively as the original animations. Therefore, we refer the reader to an openly available series of educational videos on computational mechanics [22] and to an example gallery published at https://github.com/DrSimulate/gallery, which accompanies this paper and showcases exemplary animations together with their corresponding source code.

2 | Open Source Animation Software for Visualization

In the following, we showcase three solutions for producing animations in computational mechanics. We focus our attention on freely available open source software and discuss the advantages and limitations of these methods.

2.1 | Matplotlib

Animations are essentially sequences of images that, when displayed in rapid succession, create the perception of continuous motion, as is familiar from stop-motion films. This means that animations can be generated with virtually any scientific visualization tool. One widely used open-source tool for scientific visualization is the Python library Matplotlib [18]. As Matplotlib is already familiar to many educators in computational mechanics, the entry barrier for creating scientific animations with it is relatively low.

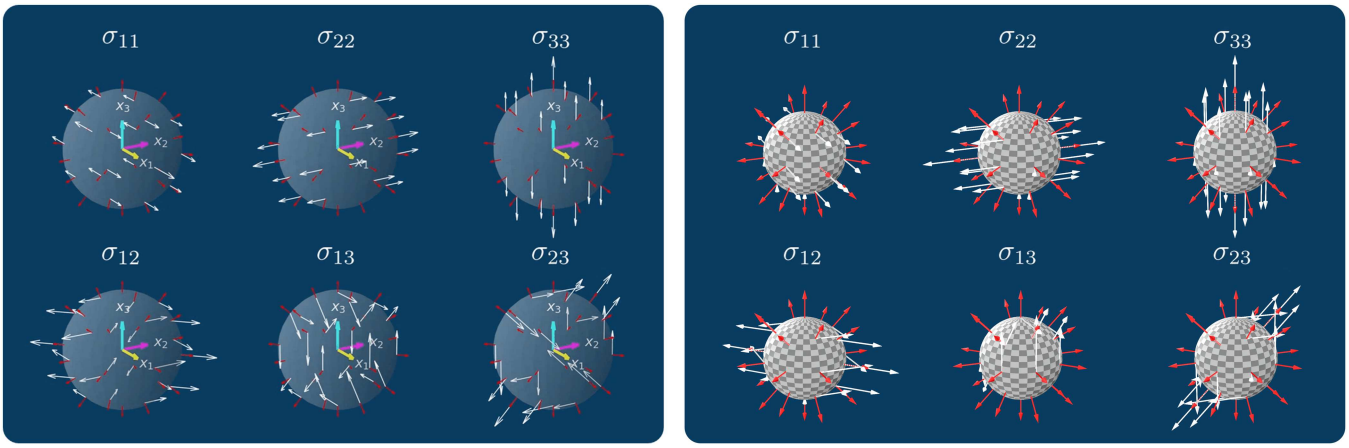
Matplotlib includes the sublibrary `matplotlib.animation`, which can be readily used to create animations. The concept is straightforward: the user controls the plotting of Matplotlib figures in each frame with the `animate()` routine, and the `FuncAnimation()` routine then combines the individual frames into a rendered video. This approach can be applied, for example, to animate the continuous evolution of graphs, two-dimensional surface plots and vector fields, or even three-dimensional scenes.

To illustrate the functionality of this approach, we present an animation of the Cauchy stress tensor components σ_{ij} in a fixed Cartesian basis, showing the surface normals and corresponding traction vectors produced by each component on an infinitesimal sphere, with all other components set to zero in turn. The following code fragment illustrates how the `animate()` routine is used to specify what is plotted in each frame, and how the `FuncAnimation()` function assembles these frames into a continuous animation, which can then be saved in the desired video format. The resulting animation shows a continuous rotation of the infinitesimal spheres, giving the appearance of a three-dimensional representation (Figure 2a).

```
# full code available at https://github.com/DrSimulate/gallery
fig = plt. figure (figsize =(_figsize, _figsize),dpi = _dpi)
ax = plt. axes (projection = '3d')
def animate (frame):
  ax. cla ()
  plot_coo (ax, shift = np. zeros (3))
  ax. plot_surface (x,y,z, alpha = .2, color = COLOR0)
  plot_vector_field (ax, normal_position, normal,
  color = COLOR1)
  plot_vector_field (ax, normal_position, factor * traction,
  color = COLOR0)
  ax. view_init (elev = 20, azim = −60 + frame/2)
```

(Continues)

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



(a) Matplotlib. (b) Manim.

FIGURE 2 | Animation of the six independent stress components in a fixed Cartesian basis. Outward unit normal vectors (red) and traction vectors (white) are shown at selected points on infinitesimal volume elements. For the animated version of (a), see https://github.com/DrSimulate/gallery.

```
    return
ani = animation. FuncAnimation (fig, animate,
frames = FACTOR_FRAMES *FRAMES, interval
  = 1/0.03)
ani. save (_path + '.mov', codec = "png", dpi =_dpi, bitrate =
−1, savefig_kwargs = {"
  transparent": True, "facecolor": "none"})
```

2.2 | Manim

Next, we discuss the Python library Manim [19, 20] and demonstrate its use for visualizing problems in computational mechanics. Manim, which is short for mathematical animation, is a library specifically designed for the creation of programmatic animations. In contrast to Matplotlib, which is primarily intended for static visualizations, Manim enables the generation of dynamic, time-resolved graphics. A distinctive feature of Manim is its support for continuously morphing geometric objects into one another via the preimplemented `Transform()` routine. This feature is particularly well-suited for visualizing motion and deformation in computational mechanics. Manim also supports the rendering of three-dimensional animations. To enable a direct comparison between Matplotlib and Manim, we reproduce the previously discussed example using Manim (Figure 2b). Finally, Manim provides functionality for animating the algebraic manipulation of equations, which can be used, for example, to visually demonstrate the derivation of governing equations in mechanics or to visualize how variables change their value as functions of time or other state variables.

While a comprehensive overview of Manim's capabilities is beyond the scope of this work, we focus in the following on a selection of core functionalities that are particularly useful for visualizing problems in computational mechanics. Manim can be used to visualize functions and how they

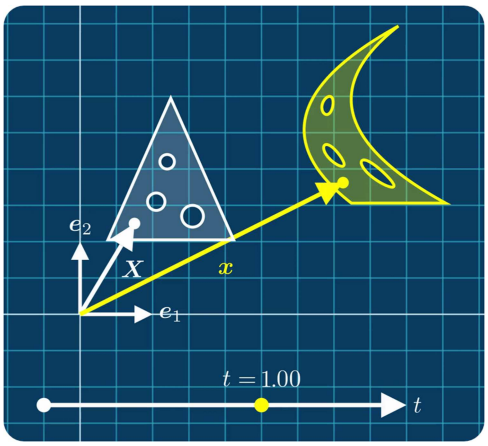


FIGURE 3 | Animation of nonlinear kinematics in continuum mechanics. A two-dimensional example is considered for simplicity. For the animated version, see https://github.com/DrSimulate/gallery.

change continuously depending on parameters. This functionality can be used, for example, to visualize a parametrized solution function in finite element analysis while continuously varying the numerical values of the parameters, as we discuss in more detail in Section 3.3. Further, the previously mentioned `Transform()` routine can be used to morph geometric shapes into one another, making it well-suited for animating object deformations. However, `Transform()` is typically restricted to a predefined set of motion mappings and does not allow full control over an object's deformation. As an alternative, Manim provides the `apply_function()` routine, which enables the deformation of an object according to a user-defined motion mapping, often denoted by $\boldsymbol{x} = \boldsymbol{\varphi}(\boldsymbol{X}, t)$ in continuum mechanics, where $\boldsymbol{X}$ and $\boldsymbol{x}$ are the undeformed and deformed points, respectively, and t is the time. The code fragment below illustrates how this routine is used to animate the nonlinear kinematics of an object (Figure 3).

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

```
# full code available at https://github.com/DrSimulate/gallery
class Kinematics2D (MovingCameraScene):
  def construct (self):
  time = ValueTracker (0.00) # scalar parameter that evolves with time
    reference = Triangle (). stretch (1.3, dim =1). move_to (2* UP + 1.25* RIGHT) #
    reference configuration
    def phi(X,t):
    x0 = (1 - 0.25* t)*X[0] + t*(X[1] -2) **2 + 2.5* t
    x1 = (1 + 0.25* t)*X[1] + 0.25* t
    x2 = X[2]
    return (x0 ,x1 ,x2) # deformed configuration
    current = always_redraw (
    lambda: reference. copy (). apply_function (lambda X: phi (X, time.
      get_value ()))
)
    self. play (time. animate. set_value (1.00),
    rate_func = linear) # animate
```

In the code above, the variable `time` is a so-called `ValueTracker()`. It is initialized with a prescribed initial value, and other objects are defined to depend on this variable. When the value of the `ValueTracker()` is updated, all dependent objects are automatically updated accordingly. This functionality enables the creation of highly customizable animations.

2.3 | Blender

Neither Matplotlib nor Manim is designed to animate objects with surface textures. However, in certain scenarios, it may be desirable to animate the motion and deformation of realistically rendered objects. This is achieved using the 3D animation software Blender [21], which enables the animation of object motion and deformation with photorealistic surface textures that accurately capture the effects of lighting and reflections. While such visual features may not directly enhance the mathematical understanding of physical processes, more realistic animations can increase students' motivation by illustrating how the mathematical concepts they learn are applied in real-world scenarios.

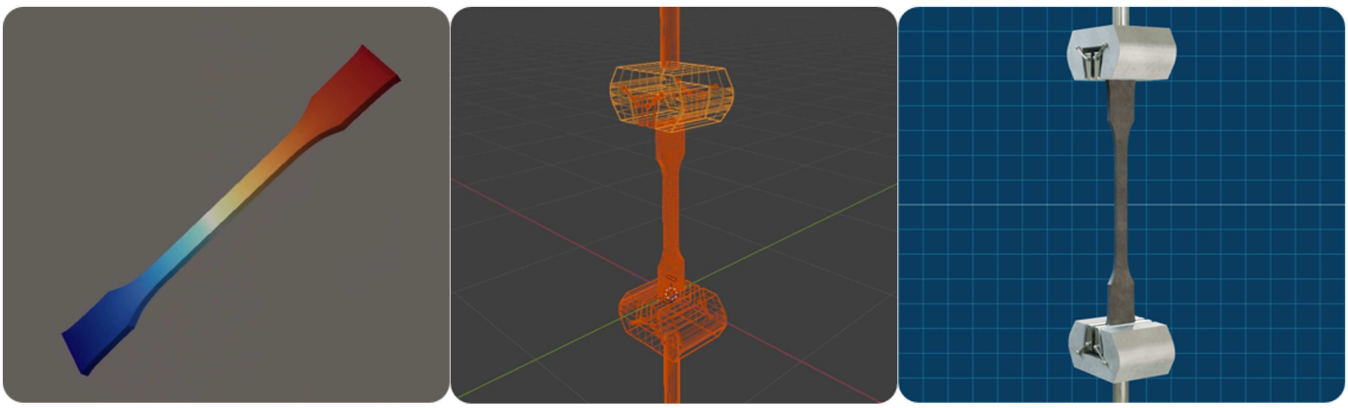
In the following, we highlight two complementary strategies for animating motion and deformation in Blender. First, we demonstrate how results obtained from a finite element analysis are imported into Blender to produce a rendered animation of simulation results. Second, we discuss how a predefined motion mapping is imposed on an object in Blender using Blender's Python programming interface.

2.3.1 | *Visualizing Finite Element Simulation Results in Blender*

Figure 4a shows the results of a finite element simulation visualized in the scientific visualization software ParaView; see Appendix A for details of the finite element simulation. ParaView can be used to inspect the mesh and visualize various fields defined on the geometry. However, it provides only limited options for advanced surface shading. For more realistic surface rendering, the results can be exported to Blender. In particular, the undeformed and deformed meshes can be exported from ParaView as a sequence of `.obj` files and subsequently imported into Blender using the add-on Stop-motion-OBJ [23], as illustrated in Figure 4b. Within Blender, the imported geometries can be rendered with realistic surface textures, lighting, and shading to produce visually appealing and physically plausible animations of the simulation results, as shown in Figure 4c.

2.3.2 | *Imposing Object Deformation in Blender*

As an alternative to importing simulated deformation results, object motions and deformations can be prescribed explicitly in Blender. In particular, Blender's Python programming interface `bpy` can be used to apply a user-defined motion mapping to an object. The following code fragment illustrates this approach.



(a) Simulation results in ParaView. (b) Deformed mesh imported in Blender. (c) Animation rendered in Blender.

FIGURE 4 | Animation of finite element simulation results in Blender. For the animated version, see https://github.com/DrSimulate/gallery.

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

```
# full code available at https://github.com/DrSimulate/gallery
mat = bpy. data. materials. new(name = "Water")
def phi(X,t=0):
  x1 = (1 + 0.75* t)*X[0]
  x2 = X[1]
  x3 = (1 - 0.25* t)*X[2] + 0.35* t*X [0]**2 - 1.5* t
  return np. array ([x1, x2, x3])
for frame in range (1, int(TOTAL_FRAMES)+1):
  t = frame/TOTAL_FRAMES
  bpy.ops. mesh. primitive_uv_sphere_add (radius =4)
  obj = bpy. context. active_object
  mesh = obj. data
  reference = np. array ([v.co [:] for v in mesh. vertices])
  for i, v in enumerate (mesh. vertices):
  v.co = Vector (phi(reference [i],t))
  for face in mesh. polygons :
  face. use_smooth = True
  visible_on_frame (obj, frame)
```

The code above generates a spherical object representing a water droplet and subsequently applies a user-defined motion mapping over a predefined set of time steps. The resulting animation is then rendered in Blender to obtain a deformable water droplet with realistic lighting and surface reflections (Figure 5).

3 | Teaching Computational Mechanics Using Visual Animations

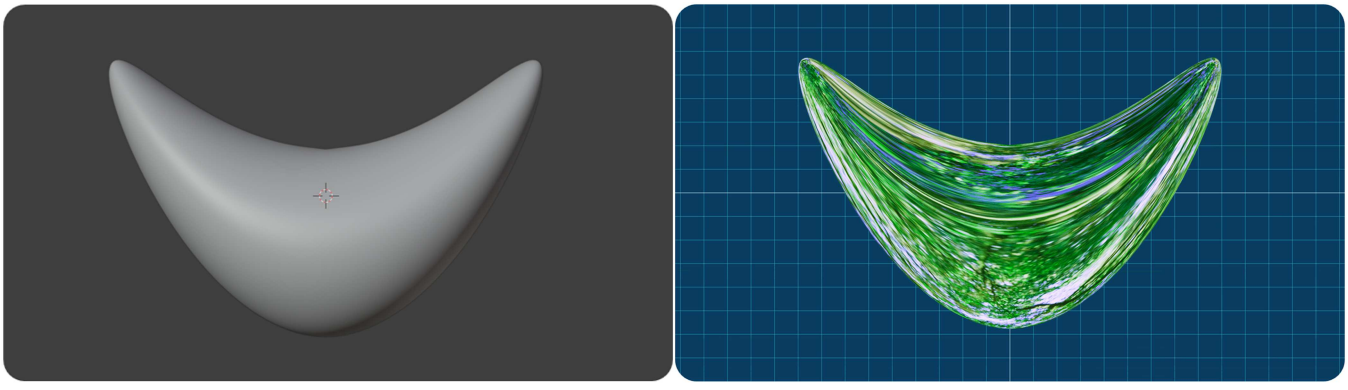
After introducing various visualization tools, we now discuss how these tools are effectively leveraged in computational mechanics. Our aim is to provide guidelines and inspiration for university instructors and science communicators, and to present examples that particularly benefit from animation.

As noted by [24], problems in computational mechanics are twofold. First, one must determine the equations governing physical processes, such as kinematic relations, balance laws, and constitutive models in continuum mechanics. Second, computational methods, such as the finite element method, are used to solve these equations numerically. From a didactic perspective, it is sensible to teach these topics in the same order: first presenting the physical problems that are being addressed and then introducing the numerical methods used to compute the corresponding solutions. Accordingly, in this work, we follow this sequence, first discussing didactic approaches and visualizations for continuum mechanics and tensor calculus, and subsequently focusing on the numerical treatment of these problems using the finite element method. All visualizations discussed below have been published in a series of educational videos on computational mechanics [22], and some of them are curated in the aforementioned example gallery.

3.1 | Continuum Mechanics

Presentations in continuum mechanics can benefit greatly from illustrative animations. A prominent example is kinematics, a purely geometric discipline concerned with the mathematical description of motion and deformation. Using the previously discussed tools, time-dependent kinematics animations can be generated in two or three dimensions. As illustrated in Figure 3, for example, the motion mapping $\boldsymbol{x} = \boldsymbol{\varphi}(\boldsymbol{X}, t)$ can be animated to visualize the associated deformation.

In two-dimensional kinematics animations, a small circle can be drawn at a material point in the reference configuration, and its deformation can be tracked over time. If the circle is assumed to be infinitesimally small, its deformation at each instant in time is described by a linear mapping. This naturally leads to the deformation gradient $\boldsymbol{F} = \frac{\partial \boldsymbol{\varphi}}{\partial \boldsymbol{X}}$, which relates undeformed and deformed line elements via $\mathrm{d}\boldsymbol{x} = \boldsymbol{F}\mathrm{d}\boldsymbol{X}$. The latter relation can be readily visualized by illustrating undeformed line element vectors and their transformation into deformed line element vectors for specific examples of the deformation gradient. In Figure 6, a unit circle and several line elements are transformed by an exemplary deformation gradient in a Cartesian basis. Analogous animations can also be created using a unit square. The freedom to choose arbitrary shapes for infinitesimal volume elements (or area elements in the two-dimensional case) can be used to sensitize students to the local linearity of the deformation gradient mapping $\mathrm{d}\boldsymbol{x} = \boldsymbol{F}\mathrm{d}\boldsymbol{X}$. Furthermore, animations of deforming infinitesimal elements can be used to illustrate that, while the deformation gradient contains no information about translation, it does encode information about



(a) Deformed mesh in Blender. (b) Animation rendered in Blender.

FIGURE 5 | Animation of a deforming water droplet in Blender using a predefined motion mapping. For the animated version, see https://github.com/DrSimulate/gallery.

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

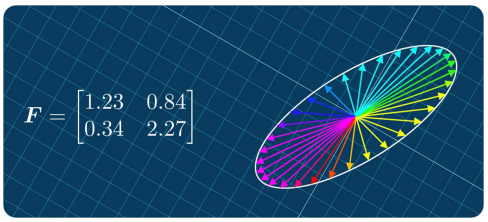


FIGURE 6 | Animation of the effect of the deformation gradient tensor on a set of undeformed line element vectors on the unit circle in a Cartesian basis. A two-dimensional example is considered for simplicity. For the animated version, see https://github.com/DrSimulate/gallery.

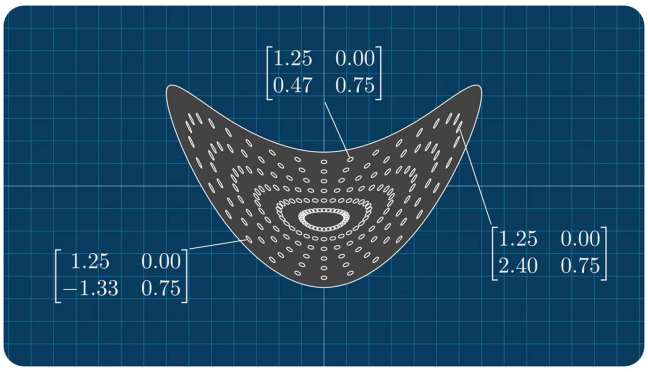


FIGURE 7 | Animation of nonlinear kinematics in continuum mechanics. The deformation gradient tensor field is visualized by animating the deformation of infinitesimal volume elements (or area elements in the two-dimensional case) at selected points. A two-dimensional example is considered for simplicity. For the animated version, see https://github.com/DrSimulate/gallery.

rotation. This observation naturally motivates the right polar decomposition of the deformation gradient $\boldsymbol{F} = \boldsymbol{RU}$, which can be visualized by first illustrating the deformation induced by the right stretch tensor $\boldsymbol{U}$, followed by the rotation induced by the tensor $\boldsymbol{R}$. Subsequently, additional measures of deformation can be introduced based on the stretch tensor, including the right Cauchy–Green tensors $\boldsymbol{C}$ and the Green–Lagrange strain tensor $\boldsymbol{E}$, and their relationship to the infinitesimal strain tensor $\boldsymbol{\varepsilon}$ can be discussed. The interpretation of the components of all these deformation tensors in a fixed basis can be conveyed by animating the deformation of infinitesimal volume elements while continuously varying the components. Finally, tensor fields can be visualized by distributing many such volume elements throughout a deforming body and animating their evolution. For example, Figure 7 illustrates a deformation gradient tensor field in which the components of $\boldsymbol{F}$ vary in space and time. The resulting deformation is visualized by animating the corresponding deformation of the infinitesimal volume elements. Analogous animations can be generated for other deformation tensors.

Fundamental to the understanding of continuum mechanics is the concept of the Cauchy stress tensor $\boldsymbol{\sigma}$ and Cauchy's theorem $\boldsymbol{t} = \boldsymbol{\sigma n}$, which establishes a relationship between the traction vector acting on a surface and the outward unit normal vector of that surface. Cauchy's theorem can be visualized by considering a small area element within a deformed body and illustrating both its outward unit normal and the corresponding traction vector. By varying the orientation of the area element, it can be demonstrated

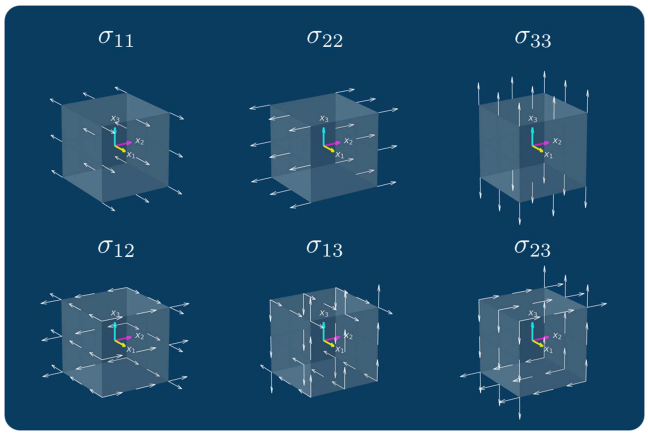


FIGURE 8 | Animation of the six independent stress components in a fixed Cartesian basis. Traction vectors are shown at selected points on infinitesimal volume elements. For the animated version, see https://github.com/DrSimulate/gallery.

that the stress tensor encapsulates the tractions acting on all possible surface orientations. This observation explains why stress is represented as a second-order tensor and is therefore fundamentally different from traction, which is a vector that represents force per unit area and is typically introduced in elementary mechanics. The components of the Cauchy stress tensor in a fixed basis can be visualized by setting all but one independent component to zero and plotting the resulting tractions acting on an infinitesimal element, as shown in Figure 2. Visualizing the tractions resulting from a stress tensor on an infinitesimal sphere has the advantage that the visualization is invariant to the choice of coordinate system. However, the shape of the infinitesimal element is arbitrary. As illustrated in Figure 8, choosing an infinitesimal cube with outward normal vectors aligned with the basis vectors shows that each column of the Cauchy stress tensor can be interpreted as the traction acting on the corresponding faces of the cube. This representation also clarifies the meaning of the two indices of the stress tensor in a Cartesian basis, one indicating the side of the cube and the other the component of the traction. For visualizations in other coordinate systems, we refer to [25].

Finally, the previously introduced kinematics animations can be extended to animate the deformation of area element vectors and Nanson's formula. In [22], an infinitesimal area element at a material point is visualized during material deformation, and the relation between the undeformed and deformed area elements is established using the determinant and the cofactor of the deformation gradient. This provides an intuitive introduction to alternative stress measures, whose associated tractions on infinitesimal area elements can be visualized. For example, Figure 9 shows undeformed and deformed area elements with their respective surface normals and tractions for a given state of deformation and stress at a material point. The Cauchy stress $\boldsymbol{\sigma}$ gives the current traction if multiplied by the normal $\boldsymbol{n}$ of the deformed area element. Multiplying this traction by the area of the deformed element $\mathrm{d}a$ yields the current force $\mathrm{d}\hat{\boldsymbol{f}} = \boldsymbol{\sigma n}\mathrm{d}a$. If multiplied by the normal $\boldsymbol{N}$ of the undeformed area element, the first Piola-Kirchhoff stress $\boldsymbol{P}$ gives an auxiliary current traction, which after multiplication by the area of the undeformed element $\mathrm{d}A$ yields the current force $\mathrm{d}\hat{\boldsymbol{f}} = \boldsymbol{P}\boldsymbol{N}\mathrm{d}A$. Finally, the second Piola-Kirchhoff stress $\boldsymbol{S}$ gives an auxiliary reference traction if multiplied by the normal of the undeformed area element. Multiplying this reference traction by the

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

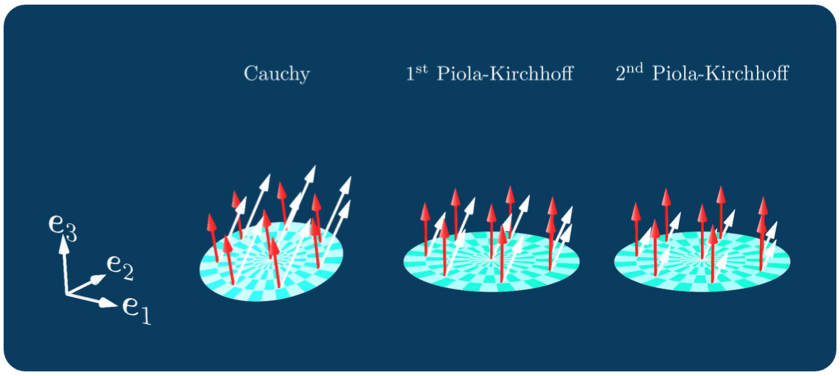


FIGURE 9 | Illustration of undeformed and deformed area elements with surface normals (red) and tractions (white) corresponding to the Cauchy stress (left), the first Piola–Kirchhoff stress (center), and the second Piola–Kirchhoff stress (right). Animating the rotation of the area elements highlights that stress tensors encode the tractions acting on arbitrarily oriented surfaces. For the animated version, see https://github.com/DrSimulate/gallery.

area of the undeformed element yields the reference force $\mathrm{d}\hat{\boldsymbol{F}} = \boldsymbol{S}\boldsymbol{N}\mathrm{d}A$, which is related to the current force by $\mathrm{d}\hat{\boldsymbol{F}} = \boldsymbol{F}^{-1}\mathrm{d}\hat{\boldsymbol{f}}$. Animating a rotation of the area elements, while updating the tractions accordingly, illustrates that stress tensors encode the tractions acting on arbitrarily oriented surfaces.

After introducing the kinematic equations and an appropriate stress measure, constitutive models can be defined at the material point. When teaching different material modeling approaches, it is important to emphasize that most models are phenomenological and based on empirical observations, in contrast to, for example, the kinematic equations. Demonstrations using everyday objects can effectively illustrate this point. For instance, it is straightforward to convince students that most materials exhibit a higher resistance to changes in volume than to changes in shape. This observation naturally motivates the separation of volumetric (shape-preserving) and isochoric (volume-preserving) deformation, such as in the Flory split. This decomposition can be animated at the level of an infinitesimal volume element to enhance intuitive understanding. In the simplest case of linear isotropic elasticity under small deformations, the volumetric-deviatoric split leads to the constitutive model $\boldsymbol{\sigma} = 3K\ \mathrm{vol}(\boldsymbol{\varepsilon}) + 2G\ \mathrm{dev}(\boldsymbol{\varepsilon})$, where K and G are the bulk and shear moduli, and $\mathrm{vol}(\boldsymbol{\varepsilon}) = \frac{1}{3}\mathrm{tr}(\boldsymbol{\varepsilon})\boldsymbol{I}$ and $\mathrm{dev}(\boldsymbol{\varepsilon}) = \boldsymbol{\varepsilon} - \mathrm{vol}(\boldsymbol{\varepsilon})$ denote the volumetric strain tensor and deviatoric strain tensor, respectively. In this model, the differing resistance to volumetric and shape changes is directly reflected in their respective contributions to the stress tensor.

Finally, standard arguments can be used to derive the balance of linear momentum. Together with suitable boundary conditions, these relations yield a boundary value problem posed on the body and its boundary, motivating the introduction of numerical methods such as the finite element method discussed in Section 3.3. The solutions of these boundary value problems can be visualized, for example, using Blender (see Figure 4).

3.2 | Tensor Calculus

Many lectures and textbooks on continuum mechanics begin by introducing tensor calculus before presenting the governing equations of continuum mechanics and the physical problems that these equations aim to describe. From a mathematical perspective, this approach is rigorous and logically coherent. However, from a didactic standpoint, the order of presentation may warrant reconsideration. Continuum mechanics has numerous practical applications that can be directly illustrated to students, thereby providing concrete motivation for the theory. Moreover, when the governing equations are introduced and analyzed in a fixed Cartesian basis, the concepts of tensor calculus are often of secondary importance. Instead, continuum mechanics concepts may first be introduced using a fixed Cartesian basis, and once the underlying theory and its physical motivation are established, the nature of tensors and their distinction from matrices familiar from linear algebra can be clarified.

Here, we focus on tensors expressed in a Cartesian basis [24] and avoid a general presentation in arbitrary coordinate systems for simplicity [26]. The nature of tensors can be illustrated to students by animating the effect of the deformation gradient tensor on a set of line element vectors in a Cartesian basis, as shown in Figure 6. Subsequently, the Cartesian basis can be changed, and the resulting change in the components of the deformation gradient tensor can be observed. This animation helps students understand that the tensor components depend on the choice of coordinate system, whereas the tensor itself and its physical interpretation remain unchanged under a change of basis. This invariance is reflected by the unchanged deformation of the unit circle in Figure 6.

The same visualization can also be used to illustrate why certain mathematical definitions and operations that apply to matrices should not be transferred analogously to tensors. For example, while diagonal matrices are commonly defined, there is no meaningful notion of a diagonal tensor. A diagonal second-order matrix is characterized by vanishing off-diagonal components in a particular basis. For a second-order tensor, however, even if all off-diagonal components vanish in one coordinate system, this need not be the case in another. This observation explains why tensor calculus often introduces the concept of isotropic tensors, whose components are equal on the diagonal and zero off the diagonal. This property is invariant under a change of basis and therefore constitutes a proper definition for a tensor.

3.3 | Finite Element Method

After introducing the governing equations of continuum mechanics, the remaining challenge is their solution.

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

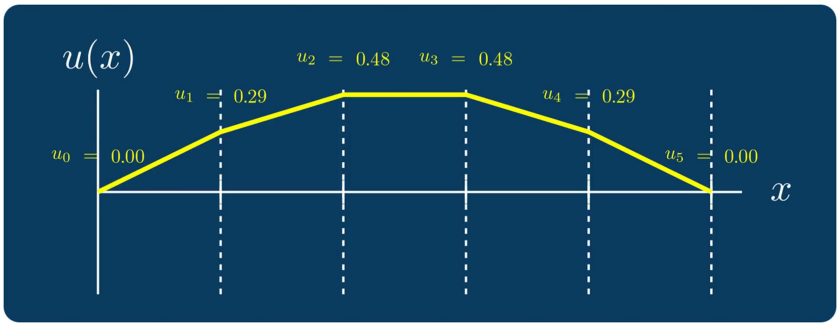


FIGURE 10 | Animation of a parameterized solution function in finite element analysis. A one-dimensional example is considered for simplicity. For the animated version, see https://github.com/DrSimulate/gallery.

Analytical solutions are generally limited to simplified models and idealized geometries. In contrast, numerical methods such as the finite element method provide a flexible framework for solving these equations on complex domains.

Computer animations can support the teaching of finite elements and enhance students' intuitive understanding. For example, the concept of discretization can be explained intuitively through animated visualizations. For simplicity, we consider a one-dimensional and time-independent example. The unknown solution function of a boundary value problem, denoted by $u(x)$, can be used to illustrate the concept of infinite-dimensional spaces. The function $u(x)$ cannot be expressed using a finite-dimensional basis, since it is unknown at infinitely many points x in the region of interest. In other words, it possesses infinitely many degrees of freedom. Computers, however, are not designed to solve problems with infinitely many degrees of freedom. This observation naturally motivates the concept of discretization, in which the solution function is approximated by a parameterized function. This parameterized function depends on a finite number of degrees of freedom and is therefore an element of a finite-dimensional space. In finite element analysis, the parameterized function is assumed to have the form

$$u(x) \approx \sum_{i=0}^{n} u_i N_i(x), \tag{1}$$

where $u_i \in \mathbb{R}$ are scalar parameters and $N_i(x)$ are finite element shape functions. The shape functions are one at the corresponding node x_i of the finite element mesh and zero at all other nodes. That is, $N_i(x_i) = 1$ and $N_i(x_j) = 0$ for $i \neq j$. In this way, it is guaranteed that the parameterized solution function equals the scalar parameters u_i at the nodes x_i. The parameterized solution function can be animated, for example, by using piecewise linear shape functions and plotting the function while continuously varying the parameters u_i, as shown in Figure 10. This helps build an intuitive understanding of the degrees of freedom associated with the parameterized solution.

Through discretization, the original infinite-dimensional unknown function is replaced by a finite set of unknown parameters. However, it remains to be answered how the unknown parameters can be computed. We consider the one-dimensional Poisson problem with Dirichlet boundary conditions as a representative example

$$\begin{aligned} u''(x) &= f(x), \forall\, x \in [0, L], \\ u(0) &= u_0, u(L) = u_L. \end{aligned} \tag{2}$$

where $f(x)$ is a source term and L denotes the length of the region of interest. A brief inspection of the problem shows that the parameterized solution function in Equation (1) cannot simply be substituted into the boundary value problem in Equation (2) and solved for the unknown parameters u_i. If the shape functions are piecewise linear, then the second derivative of the parameterized solution function is identically zero for all $x \neq x_i$. Consequently, no choice of parameters u_i can satisfy the boundary value problem. Even if the parameterized solution function is chosen to have higher regularity, substituting it into Equation (2) leads to a problem with $n + 1$ unknown degrees of freedom, while the differential equation in Equation (2) must be satisfied at infinitely many points in the region of interest. Although such problems can in principle be addressed using collocation methods, it is generally more convenient to reformulate the problem in a way that imposes fewer requirements on the parameterized solution function. This is achieved by transforming the strong formulation in Equation (2) into its weak formulation.

The weak formulation is derived by multiplying the differential equation in Equation (2) by a test function $v(x)$, integrating over the region of interest, and applying integration by parts. The resulting integral equation must hold for all sufficiently regular test functions $v(x)$ that vanish on the Dirichlet boundaries. From a didactic perspective, the derivation of the weak formulation may appear ad hoc to students in computational mechanics. It is therefore worthwhile to explain in some detail why the weak and strong formulations are equivalent under appropriate regularity assumptions, why the weak formulation is advantageous for numerical treatment, and how it can alternatively be interpreted as the optimality condition of an infinite-dimensional minimization problem.

Assuming that students are familiar with integration by parts, establishing the equivalence of the weak and strong formulations essentially reduces to explaining the fundamental theorem of the calculus of variations. Developing an intuitive understanding of this theorem constitutes a crucial first step toward comprehending the weak formulation. In simple terms, the fundamental theorem of the calculus of variations states that

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

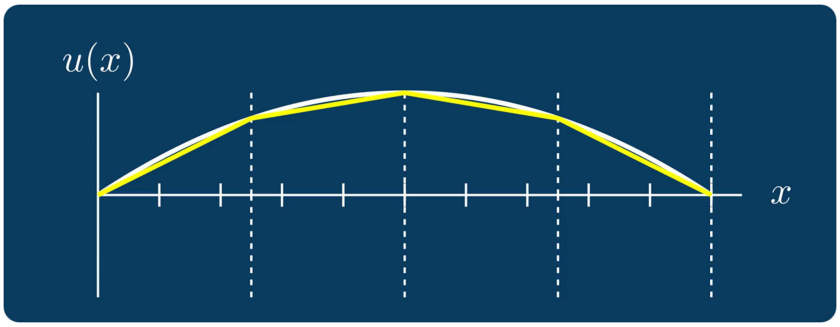


FIGURE 11 | Animation of mesh refinement in finite element analysis. The analytical solution is shown in white, and the finite element solution is shown in yellow. A one-dimensional example is considered for simplicity. For the animated version, see https://github.com/DrSimulate/gallery.

$$\int_0^L u(x)v(x)\,\mathrm{d}x = 0,\ \forall\, v(x) \quad \Leftrightarrow \quad u(x) = 0,\ \forall\, x \in [0, L], \tag{3}$$

or, stated in words, if the integral of a function $u(x)$ multiplied by an arbitrary test function vanishes for all admissible test functions, then $u(x)$ must be identically zero, and conversely. While this statement can be proven formally [27], an intuitive understanding can be gained by choosing $v(x) = u(x)$ and visualizing the integrand $u^2(x)$. Since $u^2(x)$ is nonnegative everywhere, the integral can vanish only if $u(x) = 0$. Alternatively, as we show in [22], test functions with local support can be considered, and animations can illustrate how these functions enforce the strong form locally. As the support of the test function shrinks to a point x, the strong form is recovered in a pointwise sense.

Once students have developed an intuitive understanding of the mathematical equivalence between the weak and strong formulations, the numerical advantages of the weak formulation can be discussed. In particular, the weak formulation imposes weaker regularity requirements on the parameterized solution function. Both the trial function and the test function can then be approximated using, for example, piecewise linear shape functions (see Equation 1). After standard algebraic manipulations, this leads to a system of algebraic equations that can be solved for the unknown parameters u_i.

The intuitive understanding of the weak formulation can be further strengthened by placing it in the context of the calculus of variations. In particular, the weak formulation can be interpreted as the optimality condition of an infinite-dimensional minimization problem, in which the solution function is sought to minimize a given energy functional. The arguments of energy functionals are infinite-dimensional and therefore cannot be represented using a finite basis. Consequently, classical partial derivatives are not available, and minimization cannot proceed by setting such derivatives to zero. Instead, optimality is expressed by the vanishing of the first variation, meaning that the directional derivative of the functional must be zero for all admissible perturbations $v(x)$. This condition directly yields the weak formulation. These concepts can be visualized, for example, by showing how the value of an energy functional changes continuously as the argument function varies, and by illustrating that the functional reaches a minimum only when small perturbations of the argument function do not alter its value.

For many partial differential equations in mechanics, the weak formulation can be interpreted as the principle of virtual work, or equivalently, the principle of virtual displacements. A virtual displacement is a small kinematically admissible change in the displacement field. The principle states that for any virtual displacement, the internal virtual work equals the external virtual work. This concept can be visualized by illustrating different virtual displacements and the resulting internal and external virtual work.

After computing a numerical solution of the problem in Equation (2), it can be compared to the analytical solution, if available. For example, Figure 11 shows the analytical solution of Equation (2) for a constant $f(x)$, along with the finite element solutions for different numbers of elements. Animating successive increases in the number of elements illustrates the concept of mesh convergence.

4 | Conclusions

In this work, we have presented several approaches for generating visual animations in computational mechanics using the open-source software Matplotlib, Manim, and Blender. Matplotlib provides an accessible and beginner-friendly option for creating animations. Manim, in contrast, offers greater flexibility and enables advanced three-dimensional animations with smooth transitions between geometric objects. Finally, Blender allows for the animation of object motion and deformation with realistic surface textures and lighting effects. We discussed how these animations can complement lectures in computational mechanics, with a focus on continuum mechanics, tensor calculus, and the finite element method.

There remain numerous opportunities for further exploring the use of visual animations in computational mechanics and for expanding the existing video resources [22]. In the field of continuum mechanics, an area that remains largely unexplored is the animation and didactic presentation of various material models, including small and large deformations, as well as elastic and dissipative behaviors. In this context, the thermodynamic foundations of material modeling are often challenging for students to grasp, and their introduction can appear ad hoc. Developing pedagogically sound strategies for presenting these thermodynamic concepts, ideally supported by suitable visual animations, represents an important direction for future research. With regard to tensor calculus, our focus here has been on Cartesian tensors. Extending both the theoretical

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

treatment and the accompanying visualizations to more general formulations would be a valuable avenue for future work. Similarly, while this study has emphasized one-dimensional finite element analysis, applying the presented animation tools to higher-dimensional finite elements could significantly enhance student understanding. These tools are equally applicable to other numerical methods. Finally, we anticipate that the approaches presented here will be highly useful for intuitively explaining dynamical systems and the underlying mathematics, where moving animations can provide significant pedagogical benefits.

Software

All animations presented in this contribution were created using Matplotlib (version `3.10.1`), ManimCE (version `0.18.1`), and Blender (version `4.5`).

Code and Data Availability

Code and data related to this work are partly available on GitHub [28] and partly available upon request. All video resources are publicly available on the YouTube channel *Dr. Simulate* [22]. The animations may be freely used in lecture notes and presentations with a reference to this work.

Acknowledgments

The authors utilized generative artificial intelligence to enhance the writing style in certain sections of the manuscript. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. Open Access funding enabled and organized by Projekt DEAL.

Conflicts of Interest

The author declares no conflicts of interest.

Data Availability Statement

The code and data related to this work are openly available in the example gallery at https://github.com/DrSimulate/gallery.

References

1. R. S. Palais, "The Visualization of Mathematics: Towards a Mathematical Exploratorium," *Notices of the American Mathematical Society* 46, no. 6 (1999): 647–658.

2. W. O. Wong, "Visualization of Dynamic Stress and Strain Fields in Plate Vibrations," *Computer Applications in Engineering Education* 7, no. 2 (1999): 99–106, https://doi.org/10.1002/(SICI)1099-0542(1999)7:2<99::AID-CAE3>3.0.CO;2-T.

3. S. A. Sorby and B. J. Baartmans, "The Development and Assessment of a Course for Enhancing the 3-D Spatial Visualization Skills of First Year Engineering Students," *Journal of Engineering Education* 89, no. 3 (2000): 301–307, https://doi.org/10.1002/j.2168-9830.2000.tb00529.x.

4. A. Arcavi, "The Role of Visual Representations in the Learning of Mathematics," *Educational Studies in Mathematics* 52 (2003): 215–241.

5. M. Contero, F. Naya, P. Company, J. Saorin, and J. Conesa, "Improving Visualization Skills in Engineering Education," *IEEE Computer Graphics and Applications* 25, no. 5 (2005): 24–31, https://doi.org/10.1109/MCG.2005.107.

6. J. Rogness, "Mathematical Visualization," *Journal of Mathematics Education at Teachers College* 2 (2011): 1–7.

7. S. K. Mahajan and R. S. Sonparote, "Implementation of Comparative Visualization Pedagogy for Structural Dynamics," *Computer Applications in Engineering Education* 26, no. 5 (2018): 1894–1902, https://doi.org/10.1002/cae.22024.

8. J. Schoenherr, A. R. Strohmaier, and S. Schukajlow, "Learning With Visualizations Helps: A Meta-Analysis of Visualization Interventions in Mathematics Education," *Educational Research Review* 45 (2024): 100639, https://doi.org/10.1016/j.edurev.2024.100639.

9. R. Netz, *The Shaping of Deduction in Greek Mathematics: A Study in Cognitive History* (Cambridge University Press, 1999).

10. D. Hillmayr, L. Ziernwald, F. Reinhold, S. I. Hofer, and K. M. Reiss, "The Potential of Digital Tools to Enhance Mathematics and Science Learning in Secondary Schools: A Context-Specific Meta-Analysis," *Computers & Education* 153 (2020): 103897, https://doi.org/10.1016/j.compedu.2020.103897.

11. Project Jupyter, "Jupyter Notebook," 2014, https://jupyter.org.

12. Wolfram Research Inc, "Mathematica," 1988, https://www.wolfram.com/mathematica/.

13. Desmos Studio, "Desmos Graphing Calculator," 2011, https://www.desmos.com.

14. E. Demitriadou, K. E. Stavroulia, and A. Lanitis, "Comparative Evaluation of Virtual and Augmented Reality for Teaching Mathematics in Primary Education," *Education and Information Technologies* 25, no. 1 (2020): 381–401, https://doi.org/10.1007/s10639-019-09973-5.

15. S. W. Crown, "Improving Visualization Skills of Engineering Graphics Students Using Simple JavaScript Web Based Games," *Journal of Engineering Education* 90, no. 3 (2001): 347–355, https://doi.org/10.1002/j.2168-9830.2001.tb00613.x.

16. J. Rimoli, "WebGL Player: Truss Me!," 2014, https://rimoli.gatech.edu/trussme/.

17. Z. Beck, B. Alpert, A. Bowman, W. R. Watson, and A. B. Tepole, "Elasticity Solver in Minecraft for Learning Mechanics of Materials by Gaming," *Biomedical Engineering Education* 4, no. 1 (2024): 129–135, https://doi.org/10.1007/s43683-023-00128-0.

18. J. D. Hunter, "Matplotlib," 2003, https://matplotlib.org/.

19. G. Sanderson, "Manim—Mathematical Animation Engine," 2018, https://github.com/3b1b/manim.

20. Manim Community Developers, "Manim Community Edition—Mathematical Animation Engine," 2020, https://github.com/ManimCommunity/manim.

21. Blender Online Community, "Blender—A 3D Modelling and Rendering Package," 2025, https://www.blender.org/.

22. M. Flaschel, "Educational Video Resources on Computational Mechanics," 2024, https://www.youtube.com/@DrSimulate.

23. J. Jensen, "Stop-Motion-OBJ," 2017, https://github.com/neverhood311/Stop-motion-OBJ.

24. A. J. M. Spencer, "Continuum Mechanics," 1980.

25. R. L. Bisplinghoff, J. W. Mar, and T. H. H. Pian, "Statics of Deformable Solids," 1990.

26. J. Synge and A. Schild, "Tensor Calculus," 1949.

27. S. C. Brenner and L. R. Scott, *The Mathematical Theory of Finite Element Methods. 15 of Texts in Applied Mathematics* (Springer New York, 2008), http://link.springer.com/10.1007/978-0-387-75934-0.

28. M. Flaschel, "Software for Visualizing Computational Mechanics," 2024, https://github.com/DrSimulate/gallery.

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Appendix A

Finite Element Simulation

In the following, we provide additional details on the finite element simulation shown in Figure 4. The simulation is implemented using FEniCSx. The geometry corresponds to a dogbone specimen, which is discretized using tetrahedral elements. The material behavior is described by an incompressible Mooney–Rivlin model with the strain energy density

$$W = C_1[I_1 - 3] + C_2[I_2 - 3] - p[J - 1],$$

where C_i are material constants, I_1 and I_2 are the principal invariants of the right Cauchy–Green tensor, J is the determinant of the deformation gradient, and p is a Lagrange multiplier used to enforce the incompressibility constraint. This material model could, for example, represent a rubber material as visualized in Figure 4c. We apply Dirichlet boundary conditions at the top and bottom of the specimen to deform it under displacement control. To solve the boundary value problem, we choose a mixed finite element formulation with Taylor–Hood elements.

10990542, 2026, 3, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/cae.70189 by Capes, Wiley Online Library on [05/05/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License